Quiz: Checked Attributes (Step 2)

Where you make your code eval*-free.*

You glance at the development plan. Your next step is refactoring add_checked_attribute and replacing eval with regular Ruby methods.

You may be wondering why the obsession with removing eval. How can add_checked_attribute be a target for a code injection attack if it's meant to be used only by you and your teammates? The problem is, you never know whether this method might be exposed to the world some time in the future. Besides, if you rewrite the same method without using *Strings of Code (141)*, it will only get clearer and more elegant for human readers, and less confusing for tools like syntax higlighters. These considerations are reason enough to go forward and drop eval altogether.

Can you refactor add_checked_attribute with the same method signature and the same unit tests but using standard Ruby methods in place of eval? Be forewarned that to solve this quiz, you'll have to do some research. You'll probably need to dig through the Ruby standard libraries for methods that can replace the operations in the current String of Code. You'll also need to manage scope carefully so that the attribute is defined in the context of the right class. (Hint: remember *Flat Scopes (83)*?)

Quiz Solution

To define methods in a class, you need to get into that class's scope. The previous version of add_checked_attribute did that by using an *Open Class (14)* inside a String of Code. If you remove eval, you cannot use the class keyword anymore, because class won't accept a variable for the class name. Instead, you can get into the class's scope with class_eval.

ctwc/checked_attributes/no_eval.rb

```
def add_checked_attribute(klass, attribute)
  klass.class_eval do
    # ...
  end
end
```

You're in the class now, and you can define the reader and writer methods. Previously, you did that by using the def keyword in the String of Code. Again, you can no longer use def, because you won't know the names of the methods until runtime. In place of def, you can use *Dynamic Methods (51)*:

```
def add_checked_attribute(klass, attribute)
  klass.class_eval do
```